

Enhancing a Scalable Lexical Search Tool with Syntactic/Semantic Analysis

Helping developers see how a code change can ripple through a system before they start implementing it.

Mahta Ghorbanian
University of Calgary | Department of Computer Science

Problem and thesis objective

Lexical search is fast, but code relationships can propagate change beyond matching words.

Objective: integrate ModCP syntactic/semantic analysis into DevSurge, then evaluate what the integration adds beyond lexical search.

If the tool misses an impact

- Required downstream changes may be missed
- Tests may not cover all affected behaviour
- The change becomes riskier to implement

If the tool flags too much

- Developers waste time sorting through noise
- Trust in the tool falls
- Planning becomes harder

Existing approaches and research problem

Can dependency analysis improve coverage while keeping false positives manageable?

Text (lexical) search

- Finds exact textual matches
- Provides the starting seed
- Cannot follow non-textual relationships



Syntactic/semantic analysis

- Follows calls, data flow, and control flow
- Expands from the selected seed
- May improve recall, but can add false positives

Syntactic analysis uses code structure; semantic analysis uses data- and control-dependency information.

Why the Mediator was the practical integration choice

Decision criteria: coupling, required refactoring, deployment overhead, and maintainability.

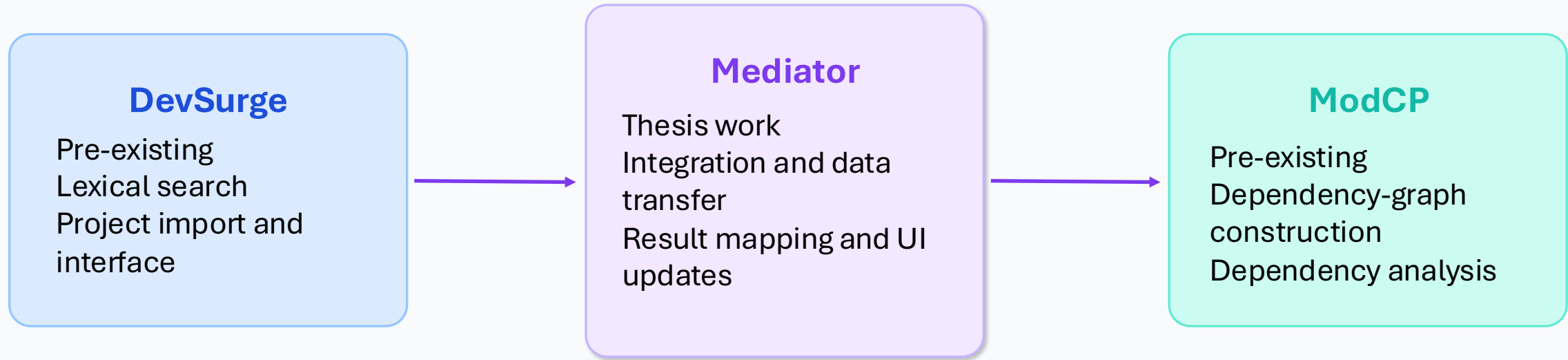
PATTERN	WHAT IT MEANS	PROJECT FIT / TRADE-OFF
Direct reference	DevSurge calls ModCP in the same process.	Low effort, but high coupling.
Plugin architecture	DevSurge loads ModCP as an extension.	Modular, but ModCP would need major changes to work through DevSurge’s plugin interface and lifecycle.
Microservice	ModCP runs as a separate network service.	Independent, but added deployment overhead.
Mediator	One component coordinates requests and results.	Best fit: loose coupling, in-process communication.

Choice: one controlled connector, while DevSurge and ModCP retain separate responsibilities.

Based on thesis section 4.1.1–4.1.4, pp. 61–64.

Contribution boundary: existing tools and thesis work

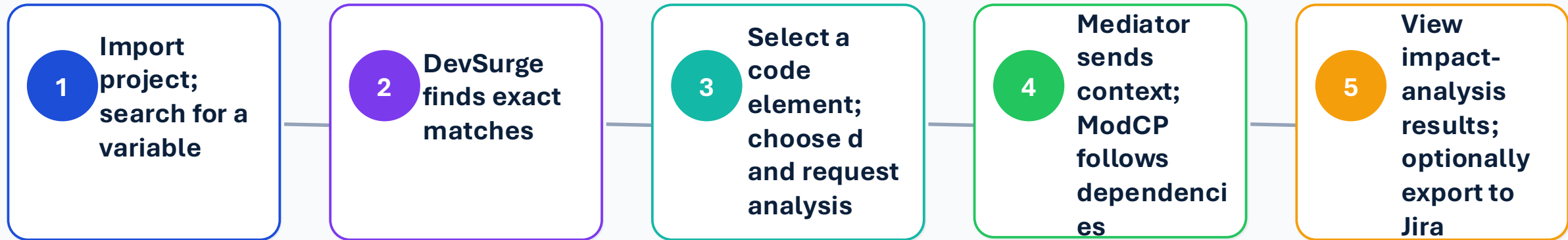
DevSurge and ModCP existed before this thesis; the integration workflow and extensions are the contribution.



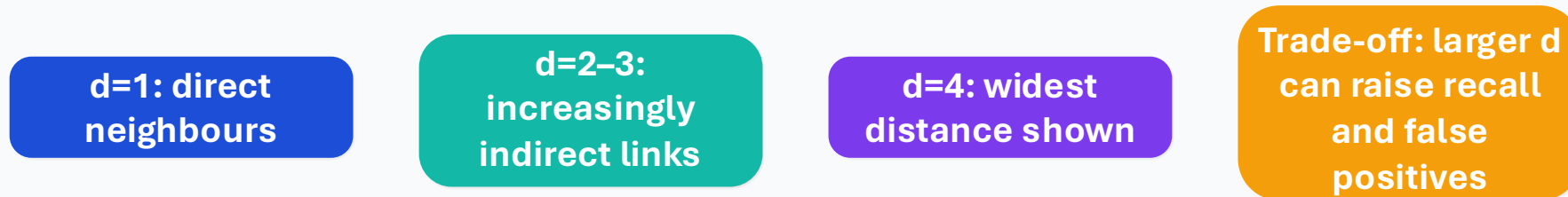
Added in this thesis: communication, caching, integrated result views, Jira export, and evaluation.

End-to-end workflow and meaning of depth d

The selected lexical match becomes the seed for dependency analysis.

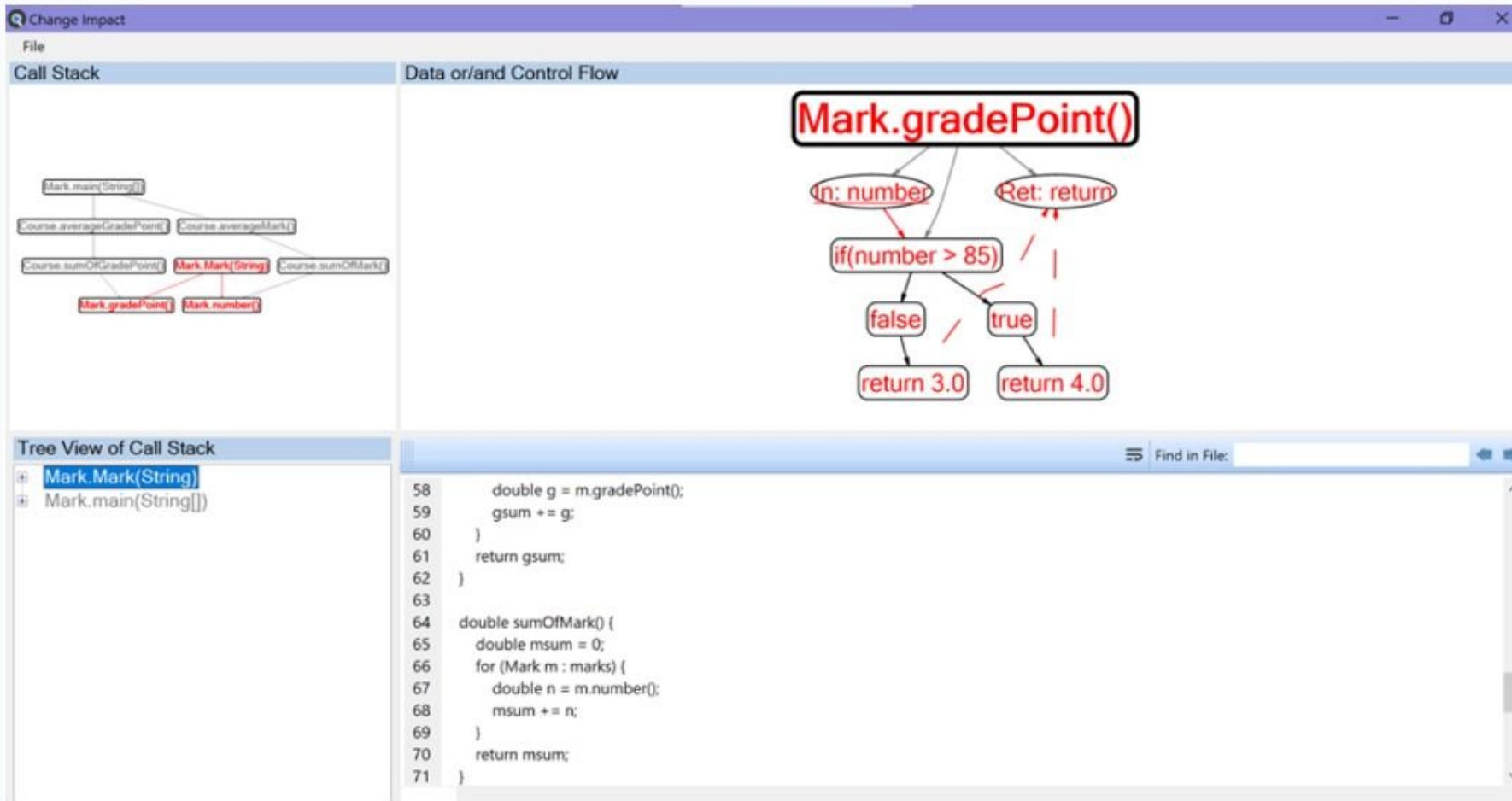


d is the maximum number of dependency steps followed from the selected seed.



How a developer explores change-impact results

The interface connects a method-level overview with detailed dependencies and source code.



Four linked views

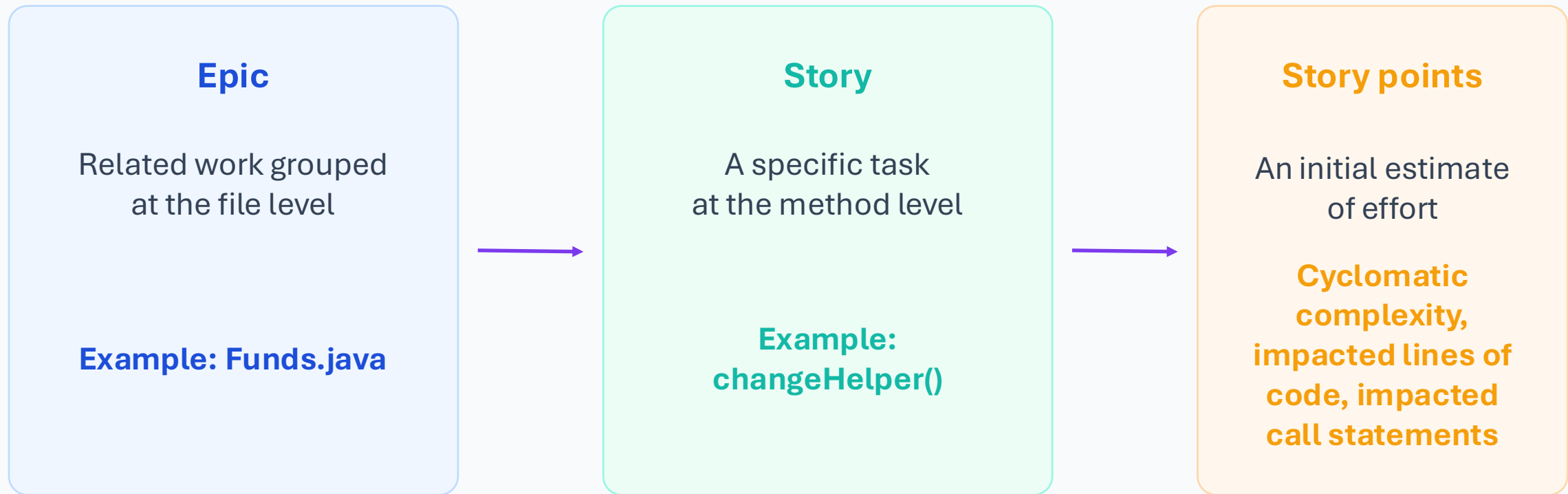
- **Call Stack Graph:**
affected methods
- **Data and/or Control Flow Graph:**
detailed dependencies
- **Tree View of Call Stack:**
hierarchy and cycles
- **Code Viewer:**
affected source lines

Together, the four views help developers trace an impact from methods to dependencies and source lines.

Based on thesis user-interface description, Figures 3.4 to 3.8.

Optional planning prototype and its evaluation scope

Impact-analysis results can be exported as Jira work items with an initial story-point value.



Evaluation scope: impact-analysis results were evaluated; story-point calculations were implemented but not compared with real project effort.

Evaluation design: results versus historical impacts

Unit of comparison: distinct source-code lines in the result set and reference set.

Dataset

- The Java Vending Machine system
- Object-oriented Java code
- Design patterns that can hide relationships in the code

Ground truth

- Historical changes and system-test evidence
- Reference lines associated with each seed-variable change
- Limitation: the result depends on completeness of this reference set

Metrics

- Precision: how many predicted impacts were correct
- Recall: how many real impacts were found
- F1 score: a balance between precision and recall

Comparison: lexical baseline versus dependency analysis at tested distances, using precision, recall, and F1.

Main result: higher F1 in one Java case study

Depth 4 was best overall for this dataset, not a universal setting.

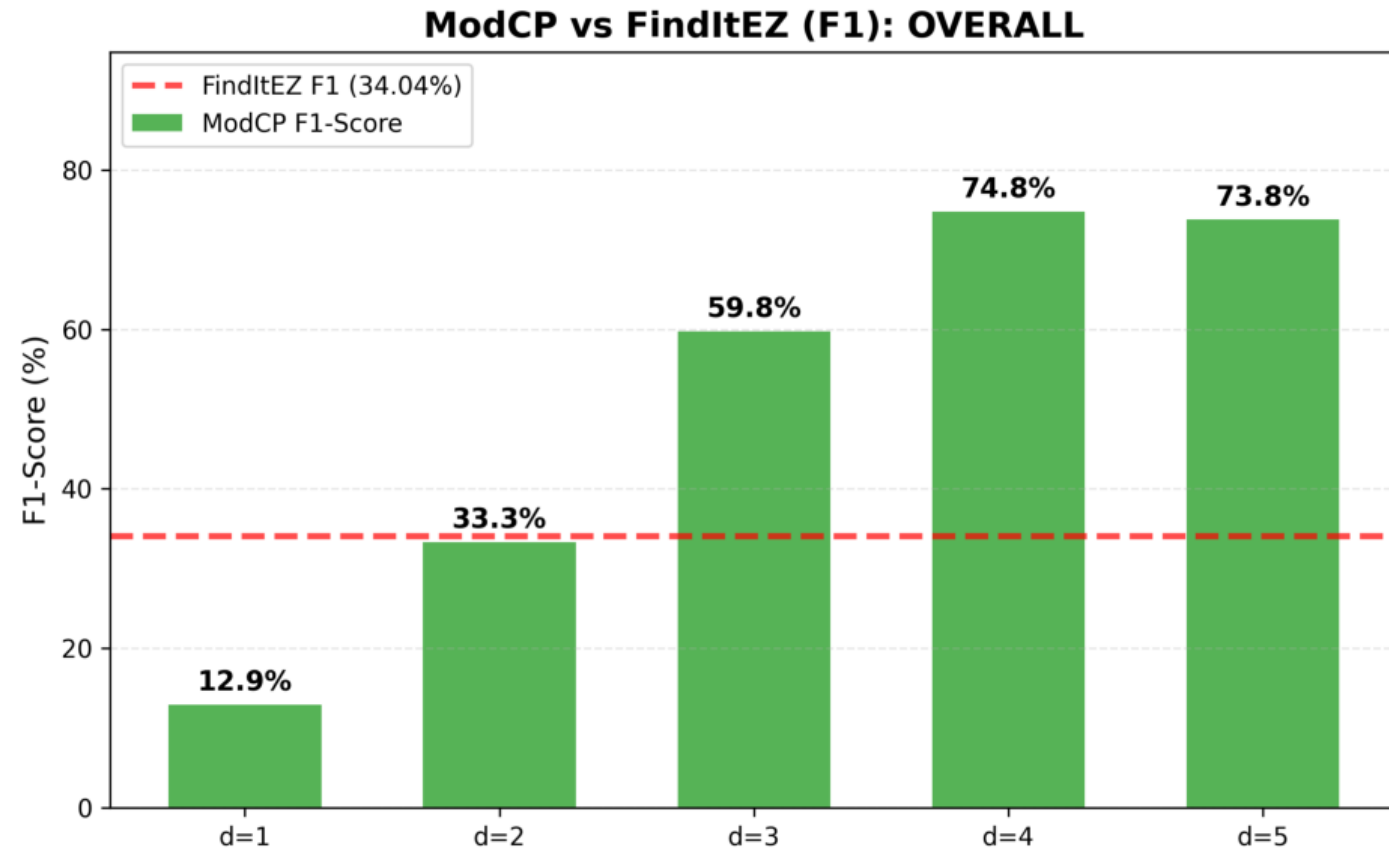


Figure 5.11: Overall F_1 -score trend vs. baseline.

Based on thesis Table 5.2 and Figures 5.10 to 5.11.

Conclusion

The integrated workflow extends lexical search with syntactic/semantic analysis and connects the results to planning.

Integration
DevSurge + ModCP

Evaluation
**Better balance of recall
and precision**

Planning support
**Jira work items and
story-point calculations**

Happy to take any questions